

V0 Plan: Daily Gmail-to-Digest Content Aggregator (Ruby CLI)

Summary

Build a Ruby command-line app in `~/bin/content_aggregator/version_three` that runs once daily (cron assumed), retrieves Gmail messages matching sender/date criteria, stores normalized records first, summarizes each message, then produces a digest body (Markdown and stdout) with top stories and links to local summary files.

Skill note: `code-skillz` was referenced in your instructions but not attached in chat, so this plan follows the style constraints you explicitly provided (clear functions, explicit names, minimal required gems).

Explicit Decisions

- Email source/auth for v0: Gmail API.
- Retrieval window logic: sender allowlist + rolling 24-hour lookback each run.
- Storage strategy: local filesystem, JSON for normalized records and Markdown for summaries.
- Summary generation: LLM API.
- Top-story selection: LLM-assisted ranking plus heuristic scoring in v0.
- Digest delivery in v0: output Markdown file and stdout only (no auto-send).
- Link targets in digest: local file paths to generated summary files.
- Deduplication key: Gmail message ID.
- CLI shape in v0: single `run` command.
- Config approach: YAML for behavior + ENV for secrets, stored locally during development.

Scope (In / Out)

- In scope:
- Daily run pipeline: `fetch -> store -> summarize -> rank -> render digest`.
- Deterministic folder/file layout for artifacts.
- Idempotent processing per run (no duplicate summary generation for same Gmail message ID).
- Out of scope (deferred):
- IMAP integration.
- Local `mbox` integration.
- Serving interface for summaries (web UI/API).
- Multi-command CLI and dry-run UX (post-v0).
- Automatic email send of digest.

Planned Interfaces and Data Contracts

- Runtime config file (YAML) with keys:
- `gmail.query_senders` (array), `gmail.lookback_hours` (default 24), `output.base_dir`, `ranking.top_n`, `llm.provider`, `llm.model`, `llm.temperature`, `llm.max_tokens`.
- ENV secrets:
- Gmail OAuth credentials/token refs, LLM API key.
- Normalized message record (JSON):
- `gmail_message_id`, `thread_id`, `sender`, `subject`, `received_at`, `snippet`, `body_text`, `labels`, `retrieved_at`, `source_query`.
- Per-item summary artifact (Markdown):
- Frontmatter: `gmail_message_id`, `summary_created_at`, `source_record_path`, `score_components`.
- Body: concise summary + why it matters + key signals.
- Run manifest (JSON):
- `run_id`, `started_at`, `ended_at`, `counts` (`fetches`, `new`, `deduped`, `summarized`), `output_paths`, `error_list`.

Execution Flow (Single run)

1. Load YAML config + ENV secrets; fail fast on missing required values.
2. Build Gmail query from allowlisted senders and 24h lookback.
3. Fetch Gmail messages via API.
4. Normalize and persist all fetched items first.
5. Deduplicate by Gmail message ID against prior processed index.
6. Summarize each new item with LLM API; persist Markdown summary.
7. Compute heuristic signals and combine with LLM ranking judgment.
8. Select top N stories.
9. Render digest Markdown with local links to summary files and print equivalent stdout output.
10. Write run manifest and status code.

File/Path Plan (New Files Only, No Overwrites)

All paths under: - `~/bin/content_aggregator/version_three`

Proposed target files for implementation phase: - `/Users/brian/dotfiles/bin/content_aggregator/version`
- `/Users/brian/dotfiles/bin/content_aggregator/version_three/lib/content_aggregator/runner.rb`
- `/Users/brian/dotfiles/bin/content_aggregator/version_three/lib/content_aggregator/config.rb`
- `/Users/brian/dotfiles/bin/content_aggregator/version_three/lib/content_aggregator/gmail_cli.rb`
- `/Users/brian/dotfiles/bin/content_aggregator/version_three/lib/content_aggregator/storage.rb`
- `/Users/brian/dotfiles/bin/content_aggregator/version_three/lib/content_aggregator/summarizer.rb`
- `/Users/brian/dotfiles/bin/content_aggregator/version_three/lib/content_aggregator/ranker.rb`
- `/Users/brian/dotfiles/bin/content_aggregator/version_three/lib/content_aggregator/digest_renderer.rb`
- `/Users/brian/dotfiles/bin/content_aggregator/version_three/config/content_aggregator.yml.example`
- `/Users/brian/dotfiles/bin/content_aggregator/version_three/spec/`

Assumptions, Defaults, Deferrals, Tradeoffs

- Assumptions/defaults:
- Cron frequency is daily (exact time/timezone TBD).
- Lookback default is 24h.
- Local artifact storage is acceptable for v0.
- LLM API is reachable during scheduled runs.
- Deferrals:
- URL-serving layer for summaries remains TBD.
- Authentication hardening and token refresh ergonomics beyond v0.
- Tradeoffs:
- Gmail API chosen over IMAP for cleaner identity and metadata fidelity, at cost of OAuth setup.
- Filesystem JSON/MD maximizes inspectability, at cost of weaker ad-hoc querying vs SQLite.
- LLM-first summarization improves quality, but introduces API cost/latency and requires fallback/error policy.

Test Cases and Acceptance Scenarios

- Unit tests:
- Query builder produces correct Gmail query for sender list + 24h window.
- Deduper skips previously processed Gmail message IDs.
- Summary parser/formatter writes deterministic Markdown metadata.
- Rank combiner behaves predictably with tie-break rules.
- Digest renderer outputs valid links to local summary files.
- Integration scenarios:
- Successful run with sample fetched messages creates records, summaries, digest, and manifest.
- Empty fetch window produces graceful “no new stories” digest.
- LLM API failure on one item logs error and continues with remaining items.
- Duplicate messages across runs do not regenerate summaries.
- Acceptance criteria:
- One command completes full pipeline without manual edits.
- Output includes top stories and local links for drill-down.
- Run manifest clearly reports success/failure counts.

Accomplished / Missing / Next Steps

- Accomplished:
- Locked core v0 architecture and key behavior decisions.
- Defined end-to-end pipeline and artifact contracts.
- Identified file-level implementation map (new files only).
- Missing (TBD):
- Exact cron schedule time and timezone.

- Initial sender allowlist values.
 - LLM provider/model and token budgets.
 - Heuristic formula details and weighting vs LLM judgment.
 - Final digest filename convention and retention policy.
 - Next Steps:
1. Confirm all TBD values below.
 2. Freeze exact file creation list.
 3. Exit plan-only mode and implement incrementally with verification after each step.

Questions You Need to Answer Before Implementation

1. What exact daily cron time and timezone should v0 use?
2. What is the initial sender allowlist (exact addresses/domains)?
3. Which LLM provider/model should v0 call, and any hard token/cost caps?
4. For top-story heuristic, which signals matter most (for example recency, sender priority, keyword match, thread activity), and relative weights?
5. What `top_n` value should the digest include by default?
6. What directory and retention policy should summary/digest artifacts use (for example keep 30 days vs indefinite)?
7. Should v0 fail hard or degrade gracefully if Gmail API or LLM API is partially unavailable?
8. Preferred digest Markdown structure (section headings/order) for your reading workflow?

Before any implementation step, I will restate the exact target file paths and wait for your literal reply: **Proceed**.

Ruby Style Contract

- This project will prioritize clear, explicit Ruby over clever or compressed syntax.
- Every runnable script will use a `main` function for orchestration and place helper methods below it.
- Function and variable names will be descriptive and concrete (for example, `get_files_to_process`, `process_file`, `file_content`).
- Control flow will stay shallow and linear where possible, with early returns over deep nesting.
- Development-stage redundancy is acceptable when it improves clarity; refactoring can follow after behavior is verified.
- Dependencies will be minimal: only required standard libraries/gems that are actively used by the script.
- Configuration values will be explicit and readable; secrets will come from environment variables, not hardcoded in source.
- Error handling for file IO, network calls, and parsing will be explicit with plain diagnostic output.

- Script structure will favor straightforward loops/counters and predictable entrypoint patterns (`main if __FILE__ == $PROGRAM_NAME`).
- No abstraction-heavy boilerplate unless and until a concrete reuse need is demonstrated.

Implementation Progress (2026-02-28)

Accomplished

- Implemented v0 runnable CLI entrypoint: `bin/content_aggregator` with optional `--config` argument.
- Implemented core pipeline modules under `lib/content_aggregator/`:
- `config.rb` for YAML + ENV-driven configuration with defaults and output path resolution.
- `gmail_client.rb` for Gmail query construction, message listing/detail fetch, body extraction, and OAuth refresh-token flow.
- `storage.rb` for run folder setup, record/summaries/digest writes, manifest writes, and processed-ID state management.
- `summarizer.rb` for per-message LLM summaries and LLM-assisted ranking selection with explicit fallback behavior.
- `ranker.rb` for heuristic scoring and combined LLM + heuristic ranking.
- `digest_renderer.rb` for Markdown digest generation with local file links.
- `runner.rb` for end-to-end orchestration: `fetch -> store -> dedupe -> summarize -> rank -> render -> manifest`.
- Added baseline tests (`minitest`) in `spec/` for:
- Gmail query logic.
- Processed-ID dedupe behavior.
- Digest link rendering.
- Completed syntax and test pass locally:
- Ruby syntax checks passed for all core files.
- 3/3 test files passed.
- Executed a smoke run with example config.
- Run produced expected `partial_failure` status because Gmail credentials are TBD.
- Manifest output confirmed at `output/runs/<run_id>/manifest.json`.

Missing (TBD / Blocked by External Inputs)

- Gmail credentials:
- `GMAIL_CLIENT_ID`, `GMAIL_CLIENT_SECRET`, `GMAIL_REFRESH_TOKEN` or `GMAIL_ACCESS_TOKEN`.
- LLM API key:
- `LLM_API_KEY`.
- Real sender allowlist values (currently placeholder addresses).
- Final LLM model name (currently `TBD_MODEL` in example config).
- Final ranking heuristic weight tuning and keyword set calibration from real data.

- Cron schedule installation details (time + timezone + deployment host context).

Next Steps

1. Provide real Gmail and LLM credentials via environment variables (do not commit).
2. Copy `config/content_aggregator.yml.example` to a local working config (for example `config/content_aggregator.yml`) and replace placeholders.
3. Run `./bin/content_aggregator -c config/content_aggregator.yml` and inspect:
 - `output/runs/<run_id>/records/`
 - `output/runs/<run_id>/summaries/`
 - `output/runs/<run_id>/digest/digest.md`
 - `output/runs/<run_id>/manifest.json`
4. Tune ranking config (`sender_priority`, `keywords`, `top_n`) against real runs.
5. Add cron entry once manual runs are stable (daily time and timezone still TBD).